

COLD START

Il primo contesto da costruire siamo noi. Prima del codice, l'ambiente.

DAY 01 · lunedì 6 luglio · 9-13 / 14-16

Il patto

Chiedere aiuto

Si chiede quando serve, senza imbarazzo.
Nessuna domanda è banale.

Camere e microfoni

Nessuno è obbligato ad accendere nulla.

La chat scritta

È un canale di pari valore, non un ripiego.
Scrivere va benissimo.

Un solo canale

Un unico luogo ufficiale dove scambiamo i file,
non tre.

Agenda • mattina

9:00	Apertura, patto e canale unico	20 min
9:20	Presentazioni, una domanda a testa	25 min
9:45	Account e accessi, uno per uno	45 min
10:30	Pausa, camere spente	15 min
10:45	File system ed estensioni visibili	30 min
11:15	File di testo puro, con Sublime	30 min
11:45	La famiglia dei formati	15 min
12:10	Markdown in pratica, avvio README	50 min

Agenda · pomeriggio

14:00	Ripresa del filo della mattina	10 min
14:10	Markdown, esercizio del README	35 min
14:45	Il terminale, le prime mosse	30 min
15:15	Pausa, camere spente	10 min
15:25	Condividere codice ed errori	25 min
15:50	Chiusura e diario di bordo	10 min



RICCARDO MANTELLI

creative technologist, artist

Creative Technologist

Interaction Design

Tangible interaction

Artificial Intelligence in Art

Installation Design

Responsive Environment Design

Cross Media Design

Media on Architecture and Public Space

Data Visualization

Computational Design

Spiritual

Jewish Kabbalah

Arabic 'Ilm al-Ḥurūf

Persian Sufism

Prophetic Culture

Artist

AI Art

Data sculpture

Data data, Data bending, Data oulipo

EM Datascape, Hertzian explorations

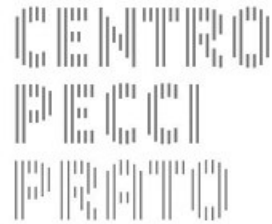
AntiGPS

Architectural mapping

Glitch art

EM injections

My field of interest and specialization areas





A selection of my clients

Google



easyJet

PALLADIO
MUSEUM

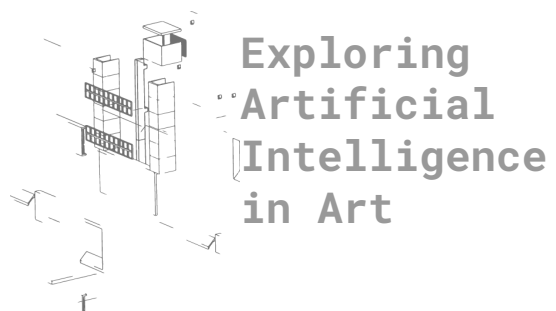
H Italian
Hospitality
Collection

CONDÉ NAST

GUESS

Miele





TRADE SCHOOL
MILANO



Freie Universität Bozen
Libera Università di Bolzano
Università Lìdia de Bulsan



DOMUS ACADEMY MILANO



Scuola Futuro Lavoro



active milano



Abadir Accademia
di Design
e Comunicazione
visiva

The university where I teach and I work to my researches

Account e accessi

01

Canale comune

Il nostro unico luogo di scambio.
Ci entri e scrivi un messaggio.

02

GitHub

Serve per Gist e per vedere
progetti reali. Lo creiamo qui se
manca.

03

Claude, Claude Code

L'account con cui parleremo agli
agenti durante il corso.

✓ **FATTO QUANDO**

Ciascuno è entrato davvero in tutti e tre, non «dovrebbe funzionare».

AI: Agentic Coding & Context Engineering · su Discord

A cosa serve

Qui trovate annunci, materiali e le schede degli esercizi.

Qui scrivete e condividete i link.

È il posto che resta tra una lezione e l'altra.

Come si lavora

Le lezioni sono su Zoom. Discord è dove si scrive:

la chat scritta è il canale principale,

nessuno è obbligato a parlare a voce.

La regola «fatto o bloccato»

Durante gli esercizi scrivete **fatto** o **bloccato** nel canale.

Così vi seguo a colpo d'occhio e vengo da chi ha bisogno.

Discord è la piazza, non l'archivio

Il codice passa da GitHub e Gist.

Qui incollate solo i link.



<https://discord.gg/xECS3GN9T>

✓ FATTO QUANDO

Tutti sono iscritti al server discord

Github

GitHub è una piattaforma web per ospitare e collaborare su codice, costruita attorno a Git, il sistema di controllo di versione creato da Linus Torvalds.

In pratica fa tre cose:

Ospita i repository. Un repository ("repo") è una cartella di progetto con l'intera storia delle sue modifiche. GitHub la conserva nel cloud, accessibile da qualsiasi macchina.

Traccia le versioni. Grazie a Git, ogni modifica è registrata come un "commit". Si può tornare indietro nel tempo, vedere chi ha cambiato cosa e quando, lavorare su linee parallele (branch) senza rompere il codice principale.

Abilita la collaborazione. Più persone lavorano sullo stesso progetto tramite pull request (proposte di modifica da rivedere e approvare), issue (segnalazioni e discussioni), e code review.

Perché ci riguarda in questo corso

GitHub è il luogo dove vive gran parte del software open source e dove gli strumenti di coding assistito da AI si integrano nativamente: si clona un repo, l'agente lavora su un branch, apre una pull request. Conoscere questo flusso è il presupposto per il coding agentico.

✓ FATTO QUANDO

Tutti sono iscritti a GitHub

GitHub Gist

Incollare codice in chat ne rompe l'indentazione e le righe. Per condividerlo si usa uno strumento apposta: un Gist diventa una pagina con un link, e conserva lo storico delle modifiche.

- 1 Vai su `gist.github.com` e accedi con GitHub.
- 2 Dai un nome al file, per esempio [esempio.py](#)
- 3 Incolla il codice nel riquadro grande.
- 4 Scegli «secret gist» e crea.
- 5 Copia il link e condividilo nel canale.

Mai chiavi, password, carte di credito o token

Non si incolla MAI una chiave, una password o un token in un Gist, un Pastebin o qualunque servizio di condivisione. Sono raggiungibili da un link: è come lasciare le chiavi di casa sulla porta.

Prima di pubblicare

rileggi e sostituisci ogni chiave con un segnaposto, per esempio `LA_TUA_CHIAVE_QUI`.

Se ti sfugge una chiave

non basta cancellare: va revocata sul servizio che l'ha emessa e rigenerata.

Un errore si copia, non si fotografa

Un errore in testo si cerca, si incolla, si fa leggere a un agente. Una foto no: sfocata, non cercabile, da ricopiare a mano.

COPIARE L'ERRORE (TESTO)

Seleziona le righe col mouse.

Windows: Ctrl+C per copiare.

Mac: Cmd+C per copiare.

Incolla nel canale, meglio in un blocco di codice.

SCREENSHOT (QUANDO SERVE)

Solo per cose visive, non per il testo.

Windows: tasto Windows + Maiusc + S.

Mac: Cmd + Maiusc + 4.

Cattura solo la parte che serve.

Il file system

Un percorso

è l'indirizzo di un file: dice in quale cartella si trova, come una via con il numero civico.

Le cartelle

contengono file e altre cartelle.

Si aprono una dentro l'altra, come scatole.

esempio di percorso

```
Documenti/  
  corso-ai/  
    day01/  
      readme.md
```

```
/Users/tawil/Documents/corso-ai/day01/
```

✓ FATTO QUANDO

Tutti sanno dire dove si trova un file e aprire la propria cartella di lavoro.

Rendere visibili le estensioni

L'estensione è la parte dopo il punto e dice che tipo di file è. Windows la nasconde: così `relazione.txt` appare come `relazione`, e non si capisce più nulla. Attiviamola subito.

WINDOWS

Apri Esplora file, scheda Visualizza, poi Mostra, e spunta «Estensioni nomi file».

MAC

Apri il Finder, menu Finder, Impostazioni, scheda Avanzate, e spunta «Mostra tutte le estensioni dei nomi».

✓ FATTO QUANDO

Tutti vedono l'estensione dei propri file.

Installazione Sublime Text

Download:

Visita il sito ufficiale `sublimetext.com`
e scarica la versione appropriata per il tuo sistema operativo.

Esecuzione:

Avvia l'installer scaricato e segui la procedura guidata di setup.

Configurazione:

Accetta i termini di licenza e seleziona il percorso di installazione desiderato.

Avvio:

Completa l'installazione e apri l'applicazione per iniziare la programmazione.

✓ FATTO QUANDO

Ognuno ha aperto un editor di testo

Il file di testo puro

Un file di testo puro contiene solo caratteri, senza formattazione nascosta: niente grassetti invisibili, niente impaginazione segreta. È l'opposto di un documento Word.

Installiamo Sublime Text (gratuito) e creiamo insieme un primo file .txt scritto a mano. Su questo poggia tutto il resto del corso.

appunti.txt

Questo è testo puro.
Solo caratteri, niente
formattazione nascosta.

✓ **FATTO QUANDO**

Ognuno ha creato e salvato un file di testo.

La famiglia dei formati

.txt

Testo senza struttura.

.md

Struttura leggera, per gli umani.

.csv

Dati a tabella, righe e colonne.

.json

Dati strutturati, per le macchine.

.yaml

Dati e configurazioni, leggibili.

.xml

Il parente più vecchio, ogni tanto spunta.

In questo corso puntiamo soprattutto su Markdown e JSON, i due evidenziati.

Markdown

È un modo per dare struttura al testo con pochi simboli semplici, restando leggibile anche prima di essere formattato. **Non è decorazione: è la lingua in cui scriveremo per parlare agli agenti.**

lo useremo per

- README.md
- AGENTS.md
- documentazione di progetto
- istruzioni per gli agenti AI
- appunti e guide operative

Il modo in cui organizziamo le informazioni cambia la qualità delle risposte: è già context engineering.

Markdown • la sintassi

Titolo

un titolo principale

Sezione

un titolo di secondo livello

****grassetto****

testo in grassetto

– voce

un elenco puntato

1. passo

un elenco numerato

[testo](url)

un collegamento

`comando`

codice dentro una frase

Markdown · Le tabelle

```
| Comando | Descrizione | Esempio |  
|-----|-----|-----|  
| Prompt | Istruzione data all'AI | "Scrivi una funzione..." |  
| Context | Informazioni aggiuntive | README.md, documentazione |  
| Output | Risultato prodotto | Codice, testo, immagini |
```

QUELLO CHE USERETE DI PIÙ

Markdown • blocchi di codice

Per un frammento di codice si aprono e si chiudono tre apici inclinati (backtick). Dopo i primi tre si può scrivere il linguaggio, così l'editor lo colora.

nel tuo file .md

```
```javascript
console.log("Hello World");
```
```

✓ **FATTO QUANDO**

Ognuno ha scritto un file Markdown con un titolo e un blocco di codice.

Il tuo primo MIONOME.md

Crea un file README.md che contenga, in ordine:

- un titolo
- una breve descrizione su di voi
- un breve elenco di esperienze pregresse
- un elenco di obiettivi

✓ **FATTO QUANDO**

Ognuno ha prodotto e salvato il proprio MIONOME.md.

Il terminale

È un modo per dare comandi al computer scrivendo, invece che cliccando. Ci serve per installare strumenti, avviare programmi e risolvere problemi.

Non dobbiamo diventare esperti: basta sapersi muovere tra le cartelle e lanciare un comando senza timore.

```
terminale
```

```
> pwd  
> mkdir progetto  
> cd progetto
```

✓ **FATTO QUANDO**

Ciascuno ha aperto il terminale e lanciato con successo almeno un comando.

Il terminale • i comandi

| AZIONE | MAC / LINUX | WINDOWS |
|---------------------------|-----------------------------|------------------------------------|
| Dove mi trovo | <code>pwd</code> | <code>pwd</code> |
| Elenco file | <code>ls</code> | <code>dir</code> • <code>ls</code> |
| Cambiare cartella | <code>cd nome</code> | <code>cd nome</code> |
| Tornare indietro | <code>cd ..</code> | <code>cd ..</code> |
| Creare cartella | <code>mkdir progetto</code> | <code>mkdir progetto</code> |
| Pulire lo schermo | <code>clear</code> | <code>cls</code> |
| Verificare Node.js | <code>node -v</code> | <code>node -v</code> |
| Verificare npm | <code>npm -v</code> | <code>npm -v</code> |
| Rimuovere un file | <code>rm file.txt</code> | <code>del file.txt</code> |
| Cancellare cartella vuota | <code>rmdir cartella</code> | <code>rmdir cartella</code> |

Cosa portiamo a casa

- un ambiente di lavoro comune e funzionante
- gli accessi risolti e il canale unico
- il primo FILE in Markdown
- le prime mosse nel terminale
- come condividere codice ed errori senza rovinarli

Domani cominciamo a parlare agli agenti.